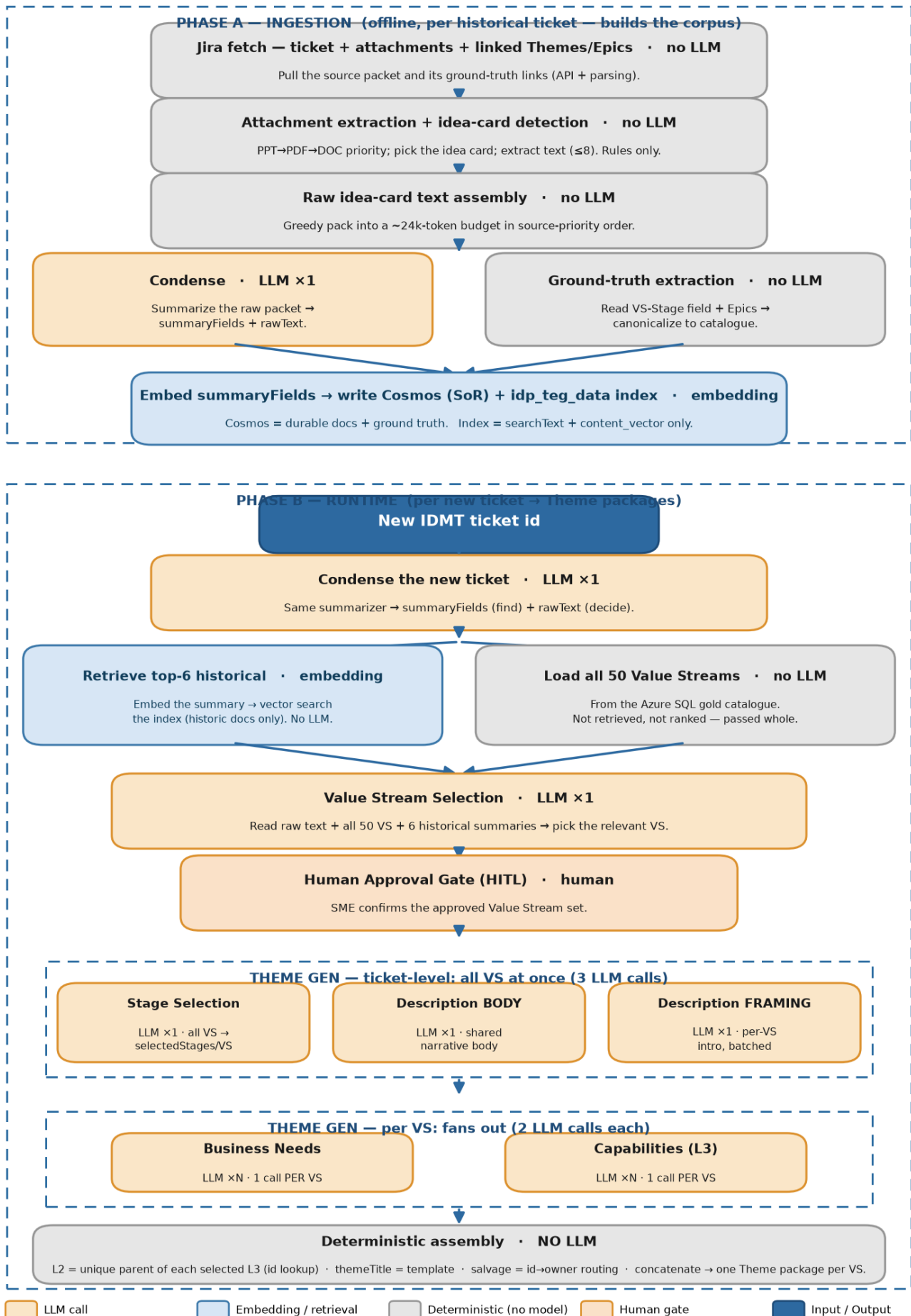


LLM invocation flow — where we call an LLM, where we don't, and why

Purpose: one place that shows the complete pipeline and, for **every** step, whether it needs an LLM. The guiding rule: **an LLM is invoked only where the task is irreducibly semantic — judgment or generation that no lookup, rule, or index can do. Everything mechanical (parsing, packing, id routing, dedup, templating, vector search) stays deterministic code.** This is the opposite of brute force: we don't ask a model to do what arithmetic or a join already does correctly and cheaply.

End-to-End Flow — where an LLM is needed (and where it is NOT)



The three kinds of step

colour	kind	example	cost
orange	LLM call — semantic judgment / generation	Condense, VS Selection, Stage Selection, Description, Business Needs, Capabilities	\$\$ + latency
blue	embedding / retrieval — a model, but not a generative call	top-6 historical vector search	cheap, no generation
grey	deterministic — pure code, no model	assembly, L2 derivation, salvage, title, packing	~free

A model appears **only** in orange (generation) and blue (embedding). Blue is *not* an LLM call — it's similarity search, which is the brute-force-efficient way to find neighbours and is far cheaper than asking an LLM to compare against a corpus.

Per-step rationale

Phase A — Ingestion (offline, per historical ticket)

step	LLM ?	why / why not brute force
Jira fetch	No	An API call + JSON parsing. Nothing to reason about.
Attachment extraction + idea-card detection	No	File-type priority (PPT→PDF→DOC) and text extraction are deterministic rules.
Raw idea-card text assembly	No	Greedy token-budget packing in a fixed priority order — arithmetic, not judgment.
Condense	Yes x1	Turning a messy, multi-format idea card into a clean summary + extracted business problem/capability is semantic compression. You cannot regex "the business problem"; meaning has to be read. One call, reused everywhere downstream so the packet is never re-processed.
Ground-truth extraction	No	The BA's answer is already in the Jira VS-Stage cascading field and the linked Epics. We read the field and canonicalize against the catalogue — a lookup, not a guess. (An LLM here would <i>fabricate</i> labels we already have authoritatively.)
Embed → write Cosmos + index	Embedding	Embedding is a model but not a generation call. It exists so retrieval is a cheap vector search later.

Phase B — Runtime (per new ticket)

step	LLM?	why / why not brute force
Condense the new ticket	Yes x1	Same reason as ingest: the new packet must be summarized (for retrieval) and assembled (for generation). "Summary to <i>find</i> , raw to <i>decide</i> ."
Retrieve top-6 historical	Embed ding	Finding similar past tickets is nearest-neighbour search , the textbook brute-force-efficient solution. Asking an LLM to scan the corpus would be slower and worse — the embedding already encodes the whole ticket cleanly (EDA: summary retrieval R 0.90 vs raw 0.84).
Load all 50 Value Streams	No	The governed catalogue is gold data in Azure SQL. We pass the whole set in — there is nothing to rank or select yet, so no model.
Value Stream Selection	Yes x1	This is the core judgment: which of 50 value streams a change touches, including implied/downstream ones . Keyword/lookup matching fails exactly on the implied picks (the hard, valuable ones). The LLM reads the raw idea card + the 50 candidates + 6 precedents and decides. Removing the count cap (recall-first) is the lever; the model, not a rule, weighs fit.
Human Approval Gate	Human	A person confirms before any generation. No model — it's the trust boundary.
Stage Selection	Yes x1 (all VS batched)	Mapping a narrow action ("automate quoting") to the lifecycle stages it runs through is semantic. The candidate stages are governed (the model can't invent), but <i>which</i> of them apply is judgment. Batched over all VS in one call — calibrated, and salvage fixes any cross-VS mislink deterministically.
Descripti on BODY	Yes x1	Writing the shared narrative — inherently generative. One VS-agnostic call per ticket.
Descripti on FRAMIN G	Yes x1 (all VS batched)	A short per-VS opening paragraph — generative. Batched so it's 1 call, not N.
Business Needs	Yes xN (1 per VS)	Generative structured prose grounded in the idea card, one consolidated document per value stream. Per-VS because each VS's stages and framing differ; batched across that VS's stages in a single call (not per stage).
Capabiliti es (L3)	Yes xN (1 per VS)	Selecting which governed L3 capabilities the work exercises, per stage, is semantic matching against candidate lists. Per VS, batched across its stages; strict-isolation prompt + salvage keep each L3 under its true stage.
L2 derivatio n	No	The clearest "no LLM" case. Each L3 capability has exactly one parent L2 in the catalogue. The L2 set is just the <i>unique parents</i> of the selected L3s — a <code>set()</code> over a lookup. Asking an LLM would add cost, latency, and a chance to get a deterministic fact wrong. Brute force (the <code>id</code> →parent map) is not just acceptable here, it's <i>correct</i> .
Theme title	No	<code>"<ticket title> – <VS name>"</code> — a string template.
Salvage (stage/L3 re-routing)	No	A stage/capability id is globally unique to one owner, so a mislinked pick is reassigned by an <code>id → owner</code> map. Deterministic correction of an LLM slip — no second LLM to "fix" the first.
Theme package assembly	No	Concatenation of the generated pieces into the response shape.

The principle, stated once

- **LLM when meaning must be produced or judged** that no table can answer: compress a messy packet (Condense), decide which value streams/stages/capabilities a change implies (Selection), or write grounded prose (Description, Business Needs).
- **Embedding when the task is "find similar"** — nearest-neighbour search is the efficient answer, not an LLM scan.
- **Deterministic everywhere else** — and deliberately so for things an LLM could *plausibly* do but shouldn't: **L2 derivation** (a parent lookup), **salvage** (id routing), **ground-truth extraction** (reading the BA's recorded answer), **titles** (templating). Using an LLM there would be brute force in the bad sense: paying a model to reproduce a fact we can compute exactly.

Call-count summary

For **N** approved value streams, one new ticket costs:

Condense	1	(LLM)
VS Selection	1	(LLM)
Stage Selection	1	(LLM, all VS batched)
Description BODY	1	(LLM)
Description FRAMING	1	(LLM, all VS batched)
Business Needs	N	(LLM, 1 per VS)
Capabilities	N	(LLM, 1 per VS)

total LLM calls	= 5 + 2N (+ 1 embedding for retrieval; L2/title/salvage/assembly = 0)	

Everything not on that list — retrieval, catalogue load, L2, salvage, title, assembly — runs without a generation call. The architecture is **lean by construction**, not by trimming.

Diagram source: `scripts/render_flow.py` (run `uv run --with matplotlib python scripts/render_flow.py`).